

前文讲解了随机法、权重随机法，本文继续讲解哈希法、一致性哈希法。

4、哈希法

哈希法的含义：使用哈希函数找到下个节点。哈希法的效果依赖哈希函数的公平性。

代码抽象：使用哈希函数，传入参数 `hashCode`，得到结果 `hashCode`。

哈希法的特点：

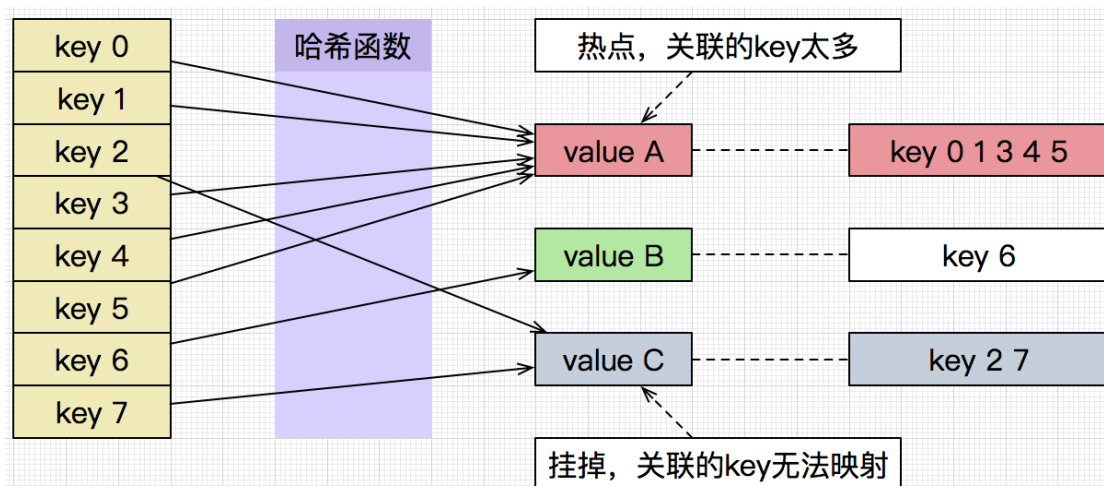
- 如果哈希函数设计合理，则公平，否则不公平。
- 公平表示哈希结果分散，利于压力分摊，下游节点负载相差不大。
- 不公平表示哈希结果集中，导致下游节点负载不均衡，某些节点压力过高，某些节点压力过低。
- 哈希函数要求简单、高效。在海量请求时，也能快速得到哈希结果。
- 常用的哈希规则：使用分组字段，使用主键字段。比如，订单表使用 `user_id` 做哈希，因为订单属于 `user`，经常用 `user_id` 查订单列表。常用 `userId`、`orderId`、`productId` 这些主键 ID 作为哈希函数的入参。

订单表的分表规则：

```
public class TableOrder {  
    public Long order_id;  
    public Long user_id;  
    public Long order_amount;  
    public Long order_pay_status;  
}  
  
// 订单表的 hash  
public static String mysqlOrderHash(TableOrder order) {  
    int index = (int) (order.user_id % 100 % 64);  
    String tableName = "order_" + index;  
    return tableName;  
}
```

哈希法的问题：

- 哈希法的核心在于哈希函数，哈希函数把某个 key 映射到固定的 value。
- 如果某个 key 的请求次数很多，则对应的 value 映射次数也很多，形成热点，导致负载不均衡。
- 如果很多 key 映射到同一个 value，也会形成热点。
- 如果某个 value 挂掉，则映射到这个 value 的很多 key，无法继续映射，功能会出错。



思考：能用随机法解决哈希法的热点问题吗？

答案是不能。

同时有读和写的负载均衡，可以用哈希法，不能用随机法。

完成写操作的数据，需要有固定的位置，才能让读操作取到之前写的的数据。随机法会导致数据的位置不固定，写、读割裂了。

比如，商品表，商品 ID=300001，写入 mysql，第一次写到分表 product_5，第二次写到分表 product_9，则商品位置不固定，读操作不知道读哪个分表。

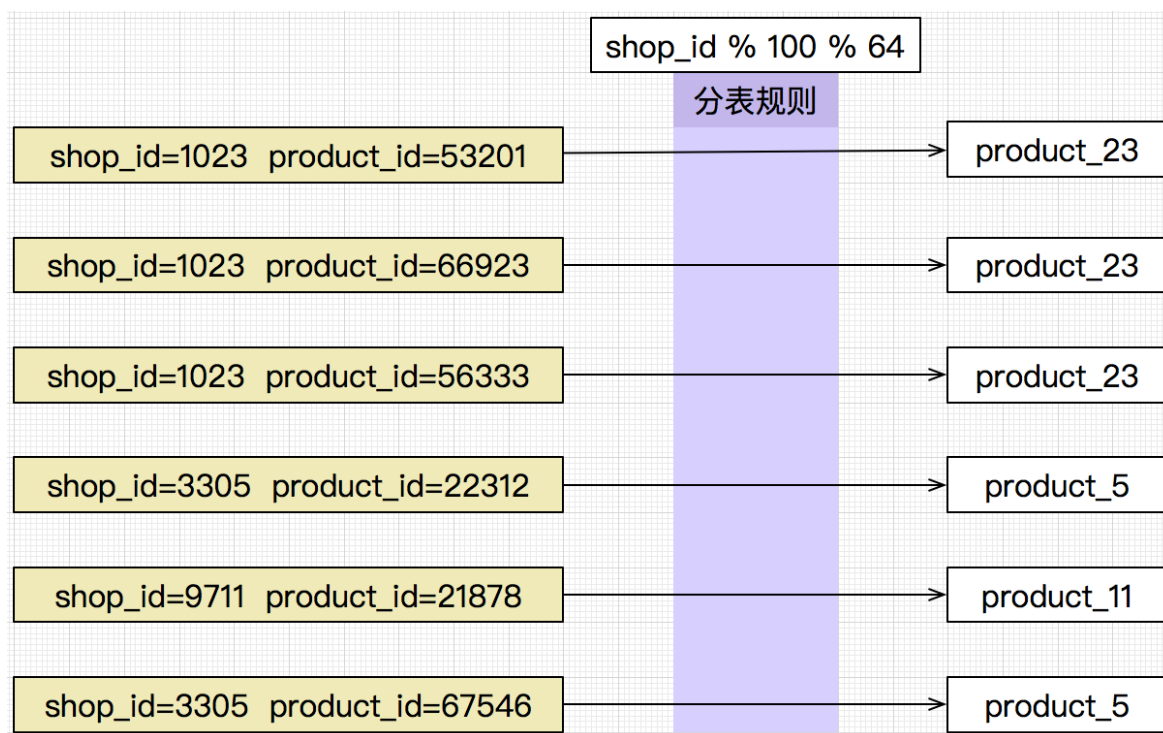
哈希法的适用场景：

- 缓存负载均衡。比如分布式 redis，有 1024 个节点。
- 数据库负载均衡。比如 mysql 订单表、用户表，使用字段+分表规则。
- 数据统计。比如用户访问量的统计，使用日期+地域作为哈希函数入参。

mysql 分表：

商品表 product，包含字段 product_id/shop_id/product_name/product_price/product_status 等。

使用 shop_id 分表，分表规则为 $\text{shop_id} \% 100 \% 64$ 。



- 分表规则使用 shop_id，读操作、写操作必须带上参数 shop_id，才能正确找到指定的分表。
- 分表规则可能导致数据热点。比如 shop_id=1023 有 50 万个商品，shop_id=3305 有 2 千个商品，shop_id=9711 有 100 个商品，则 3 个分表 product_23、product_5、product_11 的数据量相差很大。
- 对于 mysql 来说，分表数据热点常规解法是继续分表。把 64 个分表拆分为 128 个分表，同时保证某个 shop_id 的全部商品只在一个分表。

5、一致性哈希法

一致性哈希法是哈希法的增强版。

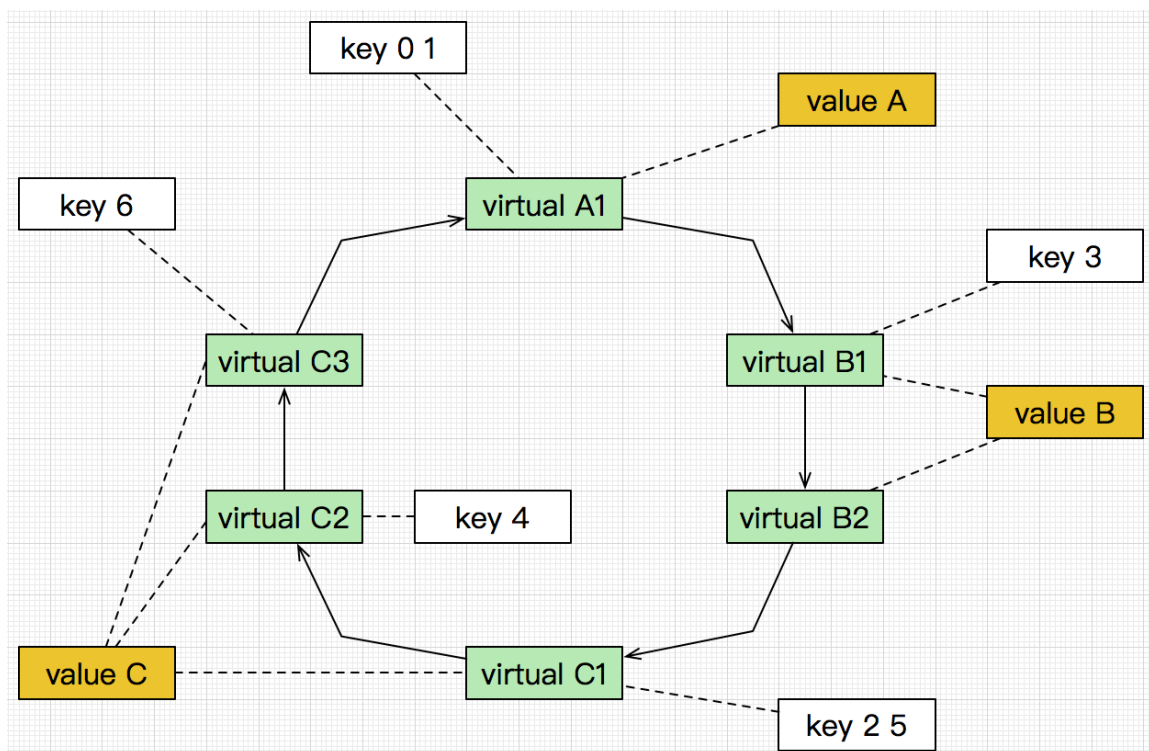
简单理解：哈希法+权重=一致性哈希法

一致性哈希法的特点：

- 节点数量增加。一致性哈希法为每个节点创建多个虚拟节点，节点总数变多，同时还可以把虚拟节点交错分布，让负载更均衡。
- 节点支持权重。可以给不同的节点设置不同的权重，权重可以用于构建虚拟节点，权重越高则虚拟节点越多，承载的流量也越多。
- 提高可用性。把全部的虚拟节点组成一个圆环，相邻的虚拟节点有前后关系。如果某个虚拟节点挂掉，则把流量自动分配给下一个有效的虚拟节点，保证流量被正常承载。如果虚拟节点恢复，则流量自动切回来。

假设，有 3 个节点 A、B、C，权重分别为 1、2、3。

- A 生成 1 个虚拟节点，A1。
- B 生成 2 个虚拟节点，B1、B2。
- C 生成 3 个虚拟节点，C1、C2、C3。
- 把全部的虚拟节点，组成圆环。
- 读写操作，作用到虚拟节点列表。



通常使用 TreeMap 实现一致性哈希法。

因为 TreeMap 查询效率高，有序性便于实现首尾相连，组成圆环，支持范围查询使得适用面更广。

这里为了简化，使用下标作为 TreeMap 的 key。

```
// 使用权重生成虚拟节点
public static List<NodeVirtual> buildVirtualNodeList(List<NodeWeight> nodeList) {
    List<NodeVirtual> nodeVirtualList = new ArrayList<>();
    for (NodeWeight node : nodeList) {
        // 使用简单策略。权重与虚拟节点数量一致。
        for (int n = 0; n < node.weight; n++) {
            NodeVirtual nodeVirtual = new NodeVirtual().setName(node.name).setRefNode(node);
            nodeVirtualList.add(nodeVirtual);
        }
    }
}
```

```

    return nodeVirtualList;
}

// 用虚拟节点生成一个环
public static TreeMap<Integer, NodeVirtual> buildNodeLoop(List<NodeVirtual> nodeVirtualList) {
    // 用红黑树，实现环。
    TreeMap<Integer, NodeVirtual> nodeMap = new TreeMap<>();
    // 遍历全部的虚拟节点，收集下标。
    for (int index = 0; index < nodeVirtualList.size(); index++) {
        Integer key = index * 100;
        nodeMap.put(key, nodeVirtualList.get(index));
    }
    return nodeMap;
}

```

查找虚拟节点可以如下实现。

floorKey(hashValue)使用范围查询，快速找到对应的虚拟节点。

```

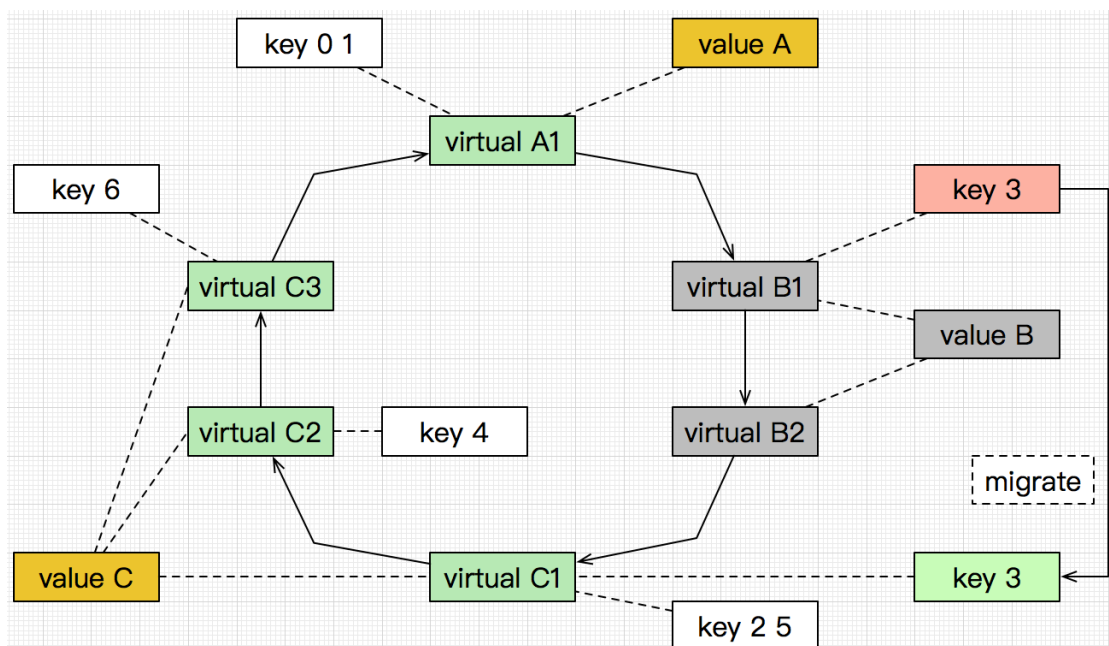
// 哈希函数。假设已经提前获得哈希值 hashValue
public static NodeVirtual balanceByHash(TreeMap<Integer, NodeVirtual> nodeLoop, int hashValue)
{
    // 找到所属的区间对应的 key
    Integer key = nodeLoop.floorKey(hashValue);
    NodeVirtual nodeVirtual = nodeLoop.get(key);
    return nodeVirtual;
}

```

功能扩展，实现故障节点迁移。

假设节点 B 出现故障，B 的虚拟节点就不可用了。一致性哈希法的环状结构，可以方便地把 B 的数据映射到下一个虚拟节点 C1。

反向思考，如果 B 恢复了，则之前临时映射到 C1 的数据也会回到 B1、B2。



改进后的实现如下。

要点在于判断节点有效性，使用 TreeMap 实现环状遍历，找到有效的虚拟节点。

```
// 哈希函数。假设已经提前获得哈希值 hashCode
public static NodeVirtual balanceByHashV2(TreeMap<Integer, NodeVirtual> nodeLoop, int
hashCode) {
    // 找到所属的区间对应的 key
    Map.Entry<Integer, NodeVirtual> start = nodeLoop.floorEntry(hashCode);
    Map.Entry<Integer, NodeVirtual> curr = start;
    while (true) {
        if (curr.getValue().refNode.valid) { // 找到了有效值
            break;
        }
        curr = nodeLoop.higherEntry(curr.getKey()); // 下一个
        if (null == curr) { // 遍历到末尾了。
            curr = nodeLoop.firstEntry(); // 头部
        }
        if (curr.getKey().equals(start.getKey())) { // 遍历完了。
            break;
        }
    }
    NodeVirtual nodeVirtual = curr.getValue();
    return nodeVirtual;
}
```